

---

ASSIGNMENT

**PROGRAMMING USING C**

**WEEK – 13**

NAME : K.Venkateshwari  
ROLL NO : 2116240801373  
DEPT & SEC :ECE F

---

# Week – 13

Given an array of numbers, find the index of the smallest array element (the pivot), for which the sums of all elements to the left and to the right are equal. The array may not be reordered.

```
/*
 * Complete the 'balancedSum' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts INTEGER_ARRAY arr as parameter.
 */

int balancedSum(int arr_count, int* arr)
{
    int totalsum = 0;
    for(int i = 0; i < arr_count; i++)
    {
        totalsum += arr[i];
    }
    int leftsum = 0;
    for(int i = 0; i < arr_count; i++)
    {
        int rightsum = totalsum - leftsum - arr[i];
        if(leftsum == rightsum)
        {
            return i;
        }
        leftsum += arr[i];
    }
    return 1;
}
```

|   | Test                                                        | Expected | Got |   |
|---|-------------------------------------------------------------|----------|-----|---|
| ✓ | int arr[] = {1,2,3,3};<br>printf("%d", balancedSum(4, arr)) | 2        | 2   | ✓ |

Passed all tests! ✓

Calculate the sum of an array of integers.

Example

numbers = [3, 13, 4, 11, 9]

The sum is  $3 + 13 + 4 + 11 + 9 = 40$ .

Function Description

Complete the function arraySum in the editor below.

```
/* Complete the 'arraySum' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts INTEGER_ARRAY numbers as parameter.
 */

int arraySum(int numbers_count, int *numbers)
{
    int sum = 0;
    for(int i=0;i<numbers_count;i++)
    {
        sum=sum+numbers[i];
    }
    return sum;
}
```

|   | Test                                                       | Expected | Got |   |
|---|------------------------------------------------------------|----------|-----|---|
| ✓ | int arr[] = {1,2,3,4,5};<br>printf("%d", arraySum(5, arr)) | 15       | 15  | ✓ |

Passed all tests! ✓

Given an array of  $n$  integers, rearrange them so that the sum of the absolute differences of all adjacent elements is minimized. Then, compute the sum of those absolute differences. Example  $n = 5$   $arr = [1, 3, 3, 2, 4]$  If the list is rearranged as  $arr' = [1, 2, 3, 3, 4]$ , the absolute differences are  $|1 - 2| = 1$ ,  $|2 - 3| = 1$ ,  $|3 - 3| = 0$ ,  $|3 - 4| = 1$ . The sum of those differences is  $1 + 1 + 0 + 1 = 3$ .

Function Description Complete the function `minDiff` in the editor below. `minDiff` has the following parameter: `arr`: an integer array

Returns: `int`: the sum of the absolute differences of adjacent elements Constraints  $2 \leq n \leq 105$   $0 \leq arr[i] \leq 109$ , where  $0 \leq i < n$

Input Format For Custom Testing The first line of input contains an integer,  $n$ , the size of `arr`. Each of the following  $n$  lines contains an integer that describes `arr[i]` (where  $0 \leq i < n$ ). Sample Case 0 Sample Input For Custom Testing STDIN Function ----- 5  
 $\rightarrow arr[]$  size  $n = 5$  5  $\rightarrow arr[] = [5, 1, 3, 7, 3]$  1 3 7 3 Sample Output 6 Explanation  $n = 5$   $arr = [5, 1, 3, 7, 3]$  If `arr` is rearranged as  $arr' = [1, 3, 3, 5, 7]$ , the differences are minimized. The final answer is  $|1 - 3| + |3 - 3| + |3 - 5| + |5 - 7| = 6$ . Sample Case 1 Sample Input For Custom Testing STDIN Function ----- 2  $\rightarrow arr[]$  size  $n = 2$  3  $\rightarrow arr[] = [3, 2]$  2 Sample Output 1 Explanation  $n = 2$   $arr = [3, 2]$  There is no need to rearrange because there are only two elements. The final answer is  $|3 - 2| = 1$ .

```
/*
 * Complete the 'minDiff' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts INTEGER_ARRAY arr as parameter.
 */
#include<stdio.h>
#include<stdlib.h>
int minDiff(int arr_count, int* arr)
{
    return(*(int*)a - *(int*)b);
}
int minDiff(int arr_count, int* arr)
{
    qsort(arr, arr_count, sizeof(int), compare);
    int totaldiff=0;
    for(int i = 1; i<arr_count; i++)
    {
        totaldiff += abs(arr[i]-arr[i-1]);
    }
    return totaldiff;
}
```

#### Syntax Error(s)

```
__tester__.c: In function 'minDiff':
__tester__.c:15:19: error: 'a' undeclared (first use in this function)
15 |     return(*(int*)a - *(int*)b);
   |               ^
__tester__.c:15:19: note: each undeclared identifier is reported only once for each function it appears in
__tester__.c:15:30: error: 'b' undeclared (first use in this function)
15 |     return(*(int*)a - *(int*)b);
   |                      ^
__tester__.c: At top level:
__tester__.c:17:5: error: redefinition of 'minDiff'
17 | int minDiff(int arr_count, int* arr)
   |     ^~~~~~
__tester__.c:13:5: note: previous definition of 'minDiff' with type 'int(int, int *)'
13 | int minDiff(int arr_count, int* arr)
   |     ^~~~~~
__tester__.c: In function 'minDiff':
__tester__.c:19:39: error: 'compare' undeclared (first use in this function)
19 |     qsort(arr, arr_count, sizeof(int), compare);
   |                                         ^~~~~~
```